

Parcial — OS

Sistemas Operativos

Tomás Rodeghiero

UNRC — Universidad Nacional de Río Cuarto

Resumen

Resolución completa y verificada del parcial “Parcial — OS” de la cátedra de Sistemas Operativos. El parcial consta de diez ejercicios mixtos (multiple-choice y abiertos) sobre: diseño de sistemas operativos (monolítico vs. microkernel), gestión de procesos, motivación de `fork+exec`, características de xv6, deadlock en pares de locks adquiridos en orden inverso, traducción de direcciones lógicas a físicas con paginación de dos niveles, diagrama de tabla de páginas para un proceso xv6, segmentación vs. paginación, paginación de un solo nivel en x86, y concepto de memoria virtual. Cada respuesta se verifica de manera independiente y se documenta el razonamiento.

Índice

1. Ejercicio 1 — Diseño de sistemas operativos	1
2. Ejercicio 2 — Gestión de procesos	3
3. Ejercicio 3 — Ventajas de <code>fork()</code> + <code>exec()</code>	4
4. Ejercicio 4 — Características de xv6	6
5. Ejercicio 5 — Evitar deadlock en par de locks invertidos	7
6. Ejercicio 6 — Traducción de direcciones lógicas a físicas	10
7. Ejercicio 7 — Diagrama del mapa de memoria de un proceso xv6	11
8. Ejercicio 8 — Segmentación y paginado	13
9. Ejercicio 9 — Una sola tabla de páginas en x86 para 4 GB	14
10. Ejercicio 10 — Concepto de memoria virtual	15
Resumen del parcial	16

1. Ejercicio 1 — Diseño de sistemas operativos

Enunciado. Marcar las respuestas verdaderas.

1.a — “Un sistema monolítico tiene los subsistemas del kernel en un mismo espacio de memoria”

Respuesta

V. En un kernel monolítico (Linux, Windows NT, xv6) todos los subsistemas (scheduler, gestor de memoria, FS, drivers, networking) se compilan en un único binario y se cargan en el mismo espacio de direcciones del kernel. La comunicación entre subsistemas es por llamadas a función directa y acceso a estructuras globales.

1.b — “En un sistema monolítico sus subsistemas se comunican por medio de mensajes”

Respuesta

F. En monolítico, los subsistemas comparten el mismo espacio de direcciones y se comunican por **llamadas a función directa** y por acceso a estructuras de datos globales (tabla de procesos, tabla de archivos, etc.). El paso de mensajes es típico del *microkernel*, no del monolítico.

1.c — “En un microkernel un subsistema no puede acceder al espacio de memoria de otros módulos”

Respuesta

V. En un microkernel los subsistemas (FS, drivers, etc.) son *procesos* en modo usuario, cada uno con su propio espacio de direcciones. La MMU impide que un servicio acceda directamente a la memoria de otro. La comunicación es vía IPC (mensajes) gestionado por el microkernel.

1.d — “Un diseño microkernel tiene como ventaja que se puede distribuir. . . ”

(diferentes componentes corriendo en diferentes computadoras en una red)

Respuesta

V. Como los servicios del SO son procesos independientes que se comunican por mensajes, esos mensajes pueden viajar tanto localmente (IPC tradicional) como a través de la red. Esto fue una ventaja explotada por Mach (base de macOS) y por sistemas distribuidos como QNX/Helios. Un servicio (e.g., un servidor de archivos) puede vivir físicamente en otra máquina y los clientes lo acceden de manera transparente.

En un monolítico esto es imposible: todos los subsistemas viven en el mismo proceso (el kernel) y dependen de acceso directo a la memoria, lo cual no es distribuible.

1.e — “Un sistema microkernel generalmente ofrece mejor rendimiento que un monolítico”

Respuesta

F. Es al revés: *en general*, los monolíticos tienen **mejor rendimiento**. La razón: en monolítico una llamada entre subsistemas es una llamada a función directa; en microkernel es un mensaje IPC con (al menos) dos cambios de modo y dos cambios de contexto, más posible copia de datos entre espacios.

A cambio, el microkernel gana en **modularidad**, **robustez** (crash de un driver no tira el kernel), **seguridad** y **distribución** (1.d).

Resumen Ej. 1

Ítem	Respuesta	Marcar como verdadera
a	V	✓
b	F	
c	V	✓
d	V	✓
e	F	

Importante

Corrección frente a la respuesta original: el ítem (d) es V. La capacidad de distribuir un microkernel en una red es justamente una de sus ventajas clásicas.

2. Ejercicio 2 — Gestión de procesos

Enunciado. Marcar las respuestas verdaderas.

2.a — “En UNIX la única llamada al sistema que crea un nuevo proceso es `fork()`”

Respuesta

V. `fork()` es la única syscall que crea un nuevo proceso (con un nuevo PID). Sus variantes `vfork()` y `clone()` en Linux son del mismo mecanismo conceptual: producen un nuevo proceso. `exec()`, en contraste, NO crea proceso: reemplaza la imagen del proceso actual.

2.b — “Un proceso *zombie* es uno tal que no puede finalizar porque no ha liberado todos sus recursos”

Respuesta

F. Un zombie *ya finalizó* (ejecutó `exit()` o fue terminado por una señal). Sus recursos (memoria, descriptors, mapeos) **ya fueron liberados**. Lo único que sigue existiendo es su *descriptor* (PCB) en la tabla de procesos del kernel, con su *exit code* guardado, esperando que el **padre** llame a `wait()/waitpid()` para recogerlo.

Sólo cuando el padre hace `wait`, el descriptor se libera y el zombie desaparece definitivamente.

2.c — “La llamada al sistema `exec(path, argv)` crea un nuevo proceso y carga el código y datos estáticos en memoria”

Respuesta

F. `exec()` no crea un nuevo proceso. Reemplaza la imagen del proceso *corriente*: mismo PID, mismo descriptor, mismo padre, pero todo el código, datos y stack se sustituyen por los del nuevo programa.

Para crear procesos se usa `fork()`. El patrón clásico es `fork + exec`: `fork` crea el hijo; `exec` reemplaza al hijo con el nuevo programa.

2.d — “Siempre hay un proceso en estado RUNNING, ya que el scheduler asume eso”

Respuesta

F. No es cierto que *siempre* haya un proceso de usuario en RUNNING:

- En xv6 (original), si la cola `RUNNABLE` está vacía, el `scheduler()` entra a un *spin* (loop) con la instrucción `hlt` o equivalente, esperando que un `wakeup` o el timer cree trabajo. No hay un “proceso idle” formal.
- En Linux y algunos SO modernos, sí existe un *idle process* (`swapper` con PID 0) que se planifica cuando no hay nadie más. Pero eso es una particularidad de la implementación, no un requisito conceptual.

El scheduler está *diseñado para tolerar* el caso de no haber procesos `RUNNABLE`. No *asume* que siempre haya algo que correr.

2.e — “Sólo puede haber un único proceso en estado RUNNING”

Respuesta

F. En sistemas multi-procesador (SMP), hay tantos procesos `RUNNING` simultáneamente como CPUs/cores haya. Cada CPU ejecuta a su proceso elegido por el scheduler. En sistemas mono-procesador sí sería V, pero la afirmación absoluta es falsa.

Resumen Ej. 2

Ítem	Respuesta	Marcar como verdadera
a	V	✓
b	F	
c	F	
d	F	
e	F	

Importante

Sólo la (a) debe marcarse como verdadera. La respuesta original tenía sin completar (d) y (e); ambas son **F**.

3. Ejercicio 3 — Ventajas de `fork()` + `exec()`

Enunciado. Describa las ventajas de que ejecutar un programa en el contexto de un nuevo proceso deba hacerse en dos pasos (`fork()` ... `exec()`). Ejemplificar.

Análisis

La división en dos syscalls (`fork`, `exec`) es una **decisión de diseño elegante** de UNIX que evita un único `spawn(prog, args, ...)` con docenas de argumentos. La idea: entre `fork` y `exec` el hijo está en una *ventana* en la que puede usar *cualquier syscall* disponible para configurar su ambiente, y luego saltar al programa final.

Ventajas concretas

1) Redirecciones de E/S por el shell. El shell hace `open/close/dup2` entre `fork` y `exec` para conectar `stdin/stdout/stderr` a archivos o pipes:

```
/* Implementacion de 'ls > out.txt' por el shell */
pid_t pid = fork();
if (pid == 0) {
    int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    dup2(fd, STDOUT_FILENO); /* fd 1 ahora apunta a out.txt */
    close(fd);
    execlp("ls", "ls", (char*)NULL);
}
```

Sin `fork/exec` separados, habría que pasarle a `ls` (o a una hipotética `spawn`) un parámetro para indicar la redirección. UNIX lo hace genérico: el shell modifica los descriptores y `ls` simplemente escribe a `stdout`.

2) Pipes entre comandos.

```
/* 'ls | wc -l' */
int p[2]; pipe(p);
if (fork() == 0) { /* hijo izquierdo: ls */
    dup2(p[1], 1);
    close(p[0]); close(p[1]);
    execlp("ls", "ls", NULL);
}
if (fork() == 0) { /* hijo derecho: wc */
    dup2(p[0], 0);
    close(p[0]); close(p[1]);
    execlp("wc", "wc", "-l", NULL);
}
close(p[0]); close(p[1]);
/* el shell espera ambos */
```

3) Cambio de ambiente. El hijo puede modificar variables de entorno con `setenv` o construirse un `envp` propio antes de `exec`, sin afectar al padre.

4) Cambio de identidad y privilegios. Programas SUID-root típicamente hacen `setuid()` para bajar sus privilegios antes de `exec`, ejecutando ya como el usuario real, no como root.

5) Cambio de directorio de trabajo. `chdir(...)` en el hijo antes de `exec` para que el nuevo programa arranque en otro directorio.

6) Configuración de signal handlers / mask. Resetear handlers a default antes del `exec`.

7) Limites de recursos. `setrlimit` (memoria máxima, tiempo de CPU, etc.) antes del `exec`.

8) Uso de fork sin exec. También permite el patrón inverso: crear procesos hijos que ejecutan el *mismo programa* (paralelismo, workers, servidores con modelo *forking*). El servidor Apache pre-fork es el ejemplo clásico.

Comparación con spawn()

Windows tiene `CreateProcess()`, un único syscall con *muchísimos* argumentos para configurar todo lo anterior. UNIX prefiere la **composición de syscalls pequeñas y ortogonales** (redirección con `dup2`, cambio de privilegios con `setuid`, etc.), y la flexibilidad de poder usar *cualquier* syscall durante la ventana del hijo.

Respuesta

La separación en dos pasos permite, en el período entre `fork` y `exec`, configurar libremente el ambiente del hijo: redirigir descriptores, conectarse a pipes, cambiar variables de entorno, bajar privilegios, cambiar el directorio de trabajo, configurar signal handlers, etc. Es la base de cómo el shell implementa redirecciones (`>`, `<`), pipes (`|`), substitución de comandos, jobs en background y demás features. Si `fork+exec` fueran un solo syscall, todo eso debería pasarse como parámetros.

4. Ejercicio 4 — Características de xv6

Enunciado. Tildar las opciones correctas sobre xv6 (código original).

4.a — “Soporta spinlock”

Respuesta

V. xv6 implementa spinlocks en `spinlock.c`: la estructura `struct spinlock` con las funciones `acquire()` y `release()`. Usa la instrucción atómica `xchg` en x86 (`amoswap.w.aq` en RISC-V) para implementar el test-and-set. Es el mecanismo de exclusión mutua fundamental del kernel.

4.b — “Soporta monitores”

Respuesta

F. xv6 *no* ofrece la abstracción de *monitor* (que en Java/C# combina un lock con variables de condición en una única construcción del lenguaje). Lo que xv6 sí ofrece son las piezas separadas: `spinlock` para exclusión mutua y `sleep/wakeup` para condition-variables-like (item 4.c), pero el programador del kernel las combina manualmente.

4.c — “El gestor de procesos tiene funciones comparables a variables de condición”

Respuesta

V. Las funciones `sleep(chan, lock)` y `wakeup(chan)` de xv6 implementan *exactamente* la semántica de variables de condición:

- `sleep(chan, lock)`: libera `lock`, pone al proceso a dormir en `chan` (estado `SLEEPING`). Al despertar, vuelve a tomar el lock. Equivalente a `pthread_cond_wait`.
- `wakeup(chan)`: despierta a *todos* los procesos dormidos en `chan` (pasa a `RUNNABLE`). Equivalente a `pthread_cond_broadcast`.

xv6 no llama a estas operaciones “variables de condición” explícitamente, pero la analogía es directa.

4.d — “No ofrece ningún mecanismo de comunicación y sincronización entre procesos”

Respuesta

F. xv6 *sí* ofrece mecanismos:

- **Pipes**: `pipe(p)` crea pipes entre procesos emparentados.
- **Archivos**: pueden usarse para comunicar procesos.
- **Signals (kill)**: aunque limitadas, xv6 implementa `kill(pid)`.
- **wait/exit**: sincronización padre-hijo a través del estado de salida.
- **Locks compartidos en filesystem**: `ilock` para inodos compartidos.

4.e — “Tiene mecanismo de detección de deadlock”

Respuesta

F. xv6 *no* detecta deadlocks. La política es “ostrich algorithm”: si los locks del kernel se toman siempre en el mismo orden global (convención del código), el deadlock no ocurrirá; si el programador comete un error de ordenamiento, xv6 simplemente se cuelga.

No hay grafo de espera, no hay timeout en `acquire`, no hay recuperación. La prevención se hace por disciplina, no por detección automática.

Resumen Ej. 4

Ítem	Respuesta	Marcar como verdadera
a	V	✓
b	F	
c	V	✓
d	F	
e	F	

5. Ejercicio 5 — Evitar deadlock en par de locks invertidos

Enunciado. Proponer una solución al código dado para evitar deadlock. Describir ventajas/-desventajas.

```
// module A // module B
acquire(&a.lock); acquire(&b.lock);
// update a.data // update b.value
```

```

acquire(&b.lock); acquire(&a.lock);
// update b.value // update a.data
release(&b.lock); release(&a.lock);
release(&a.lock); release(&b.lock);

```

Análisis: por qué hay deadlock

Los módulos *A* y *B* adquieren los mismos dos locks en **orden inverso**: *A* toma *a* y luego *b*; *B* toma *b* y luego *a*. Una traza con ambos ejecutándose en paralelo:

1. *A* ejecuta `acquire(&a.lock)`: obtiene *a*.
2. *B* ejecuta `acquire(&b.lock)`: obtiene *b*.
3. *A* ejecuta `acquire(&b.lock)`: bloquea (*B* tiene *b*).
4. *B* ejecuta `acquire(&a.lock)`: bloquea (*A* tiene *a*).

Ambos bloqueados esperando un lock que el otro retiene → **deadlock** (las 4 condiciones de Coffman se cumplen).

Solución 1 (recomendada): orden global de adquisición

Imponer un **orden total** sobre los locks que ambos módulos respeten. Por ejemplo, “siempre tomar primero el lock con menor dirección de memoria”, o “siempre primero `a.lock` y luego `b.lock`”.

```

/* Versión corregida: ambos módulos toman a.lock ANTES que b.lock */

// module A // module B
acquire(&a.lock); acquire(&a.lock); // CAMBIA: primero a
acquire(&b.lock); acquire(&b.lock); // luego b

// update a.data // update a.data
// update b.value // update b.value

release(&b.lock); release(&b.lock);
release(&a.lock); release(&a.lock);

```

Por qué funciona. Con orden global, nunca pueden formar una espera circular: el primero que tome `a.lock` es el que después podrá pedir `b.lock`, y el otro espera al primero. Como mucho hay una cadena lineal, jamás un ciclo.

Ventajas.

- **Simple** de implementar y razonar.
- **Sin penalidad de performance**: los locks se toman normalmente, sólo cambia el orden.
- **Granularidad fina**: dos hilos que toquen partes disjuntas del estado pueden ejecutar concurrentemente (sólo se serializan cuando colisionan en los mismos locks).

Desventajas.

- Requiere disciplina del programador: si en algún lugar del código alguien viola el orden, vuelve el deadlock.
- En sistemas grandes con muchos locks, mantener el orden global se complica.

Solución 2: un único lock global

Sustituir los dos locks por uno solo, que proteja todo el estado.

Ventajas.

- **Imposible** formar deadlock con un único lock.
- Trivial de implementar.

Desventajas.

- **Pérdida de concurrencia:** dos hilos que tocan partes disjuntas del estado ahora se serializan innecesariamente.
- En sistemas con alta contención, este lock se vuelve cuello de botella.

Solución 3: trylock con *back-off*

Usar `trylock` para el segundo lock; si falla, soltar el primero y reintentar.

```
while (1) {
    acquire(&a.lock);
    if (trylock(&b.lock) == 0) { /* tomado */
        break;
    }
    release(&a.lock);
    /* opcional: sleep o spin un rato antes de reintentar */
}
```

/ aqui A tiene a y b */*

Ventajas.

- No requiere coordinación global de orden.
- Funciona en escenarios donde no se sabe a priori el conjunto de locks.

Desventajas.

- Más complejo de implementar.
- Puede generar **livelock**: dos hilos que se ceden mutuamente y nunca progresan. Requiere *back-off aleatorio* para mitigarlo.
- Lecturas/escrituras parciales en estructura compartida si no se cuida con `release` oportuno.

Comparación

	Orden global	Un solo lock	Trylock + back-off
Complejidad	Baja	Mínima	Alta
Concurrencia	Buena (locks separados)	Mala (serialización)	Buena (intentos no bloquean)
Riesgo de deadlock	0 si se respeta el orden	0	0, pero hay livelock
Aplicabilidad	Cuando los locks se conocen estáticamente	Siempre	Cuando los locks son dinámicos

Respuesta

La solución estándar es **imponer un orden global de adquisición**: ambos módulos toman primero `a.lock` y luego `b.lock` (o el orden inverso, pero consistente). Esto elimina la espera circular sin sacrificar concurrencia. La desventaja es que exige disciplina del programador para mantener el orden a lo largo de toda la base de código.

6. Ejercicio 6 — Traducción de direcciones lógicas a físicas

Enunciado. Dado el mapa de memoria mostrado, determinar las direcciones físicas correspondientes a: **a)** 0x00000320 **b)** 0x00002FFA **c)** 0x00400200. También determinar el rango de direcciones lógicas que mapean al **frame 3**.

Análisis: estructura del mapa

El diagrama muestra paginación de **dos niveles** estilo x86 32 bits (4 KB páginas):

- Page directory con entradas 0, 1, 2, ...
- Page table 1 con dos entradas: 0x2000 → frame 1 y 0xA000 → frame 3.
- Page table 2 con una entrada: 0x5000 → frame 2.

Las flechas del diagrama indican:

- pgdir[0] → Page Table 1.
- pgdir[1] → Page Table 2.
- PT1[0] = 0x2000 | flags → frame 1 en físico 0x02000000.
- PT1[2] = 0xA000 | flags → frame 3 en físico 0x0A000000.
- PT2[0] = 0x5000 | flags → frame 2 en físico 0x05000000.

Codificación de direcciones (32 bits, páginas de 4 KB):

- bits 31–22: pgdir index (10 bits).
- bits 21–12: pgtable index (10 bits).
- bits 11–0: offset (12 bits).

Interpretación del PTE: el valor del PTE (e.g., 0x2000) representa el **frame number** (PPN); la dirección física del frame es $PPN \times 4 \text{ KB}$. Así: $0x2000 \rightarrow \text{físico } 0x2000 \times 0x1000 = 0x02000000$.

6.a — Dirección lógica 0x00000320

Descomposición:

$$0x00000320 = 0000_0000_00 \ 00_0000_0000 \ 0011_0010_0000$$

- pgdir = 0
- pgtable = 0
- offset = 0x320

Camino: pgdir[0] → PT1; PT1[0] = 0x2000 → frame en físico 0x02000000.

$$\text{Físico} = 0x02000000 + 0x320 = 0x02000320.$$

6.b — Dirección lógica 0x00002FFA

Descomposición:

$$0x00002FFA = 0000_0000_00 \ 00_0000_0010 \ 1111_1111_1010$$

- pgdir = 0
- pgtable = 2
- offset = 0xFFA

Camino: pgdir[0] → PT1; PT1[2] = 0xA000 → frame en físico 0x0A000000.

$$\text{Físico} = 0x0A000000 + 0xFFA = 0x0A000FFA.$$

6.c — Dirección lógica 0x00400200

Descomposición:

$$0x00400200 = 0000_0000_01\ 00_0000_0000\ 0010_0000_0000$$

- pgdir = 1
- pgtable = 0
- offset = 0x200

Camino: pgdir[1] → PT2; PT2[0] = 0x5000 → frame en físico 0x05000000.

$$\text{Físico} = 0x05000000 + 0x200 = 0x05000200.$$

6.d — Rango lógico que mapea al frame 3

Importante

El rango original [0x00400000, 0x0040BFFF] estaba incorrecto. El razonamiento dado (“pgdir[1] + pgtable[0]”) apuntaba al frame 2, no al frame 3.

Razonamiento correcto. El frame 3 se alcanza a través de pgdir[0] (que apunta a PT1) y pgtable[2] (entrada 0xA000 de PT1). Esto es *coherente* con la respuesta de 6.b: 0x00002FFA mapeó a frame 3.

Por lo tanto, las direcciones lógicas que caen en frame 3 son aquellas con:

- bits 31–22 (pgdir) = 0
- bits 21–12 (pgtable) = 2
- bits 11–0 (offset) ∈ [0, 0xFFF]

Composición:

$$\text{mínimo: } 0000_0000_00_00_0000_0010_0000_0000_0000 = 0x00002000$$

$$\text{máximo: } 0000_0000_00_00_0000_0010_1111_1111_1111 = 0x00002FFF$$

Respuesta

Direcciones físicas:

Lógica	pgdir	pgtable[idx]	Física
0x00000320	0	PT1[0] = 0x2000	0x02000320
0x00002FFA	0	PT1[2] = 0xA000	0x0A000FFA
0x00400200	1	PT2[0] = 0x5000	0x05000200

Rango lógico del frame 3: [0x00002000, 0x00002FFF] (4 KB consecutivos, accedidos vía pgdir[0] + pgtable[2]).

7. Ejercicio 7 — Diagrama del mapa de memoria de un proceso xv6

Enunciado. Diagrame el mapa (directorio + tablas de páginas) para un proceso xv6 que usa 15 KB de código, 3 KB de datos (y el stack correspondiente).

Análisis: layout que produce exec en xv6

xv6 (versión del libro) coloca en el espacio de direcciones del proceso, comenzando en 0:

1. Las páginas de **código (text)**.

2. Las páginas de **datos** (`data + bss`).
3. Una **página de guarda** con $V = 0$ (para detectar *stack overflow*).
4. Una **página de stack** (el código de `exec` en `xv6` asigna 1 página de stack utilizable, en `exec.c`: “Allocate two pages ... make the first inaccessible ... use the second as the user stack”).

Cuentas previas

- **Código (15 KB)**: $\lceil 15/4 \rceil = 4$ páginas (la última con 1 KB de *padding*). → vpns **0, 1, 2, 3**.
- **Datos (3 KB)**: $\lceil 3/4 \rceil = 1$ página (con 1 KB sin usar). → vpn **4**.
- **Guarda**: 1 página con $V = 0$. → vpn **5**.
- **Stack**: 1 página. → vpn **6**.

Total: **7 vpns**, de las cuales **6 están mapeadas** (la guarda es inválida). Todas caen en los primeros 28 KB del espacio de direcciones; todas pertenecen al primer entrada del page directory (`pgdir[0]`, que cubre los primeros 4 MB).

Diagrama del directorio + tabla de páginas

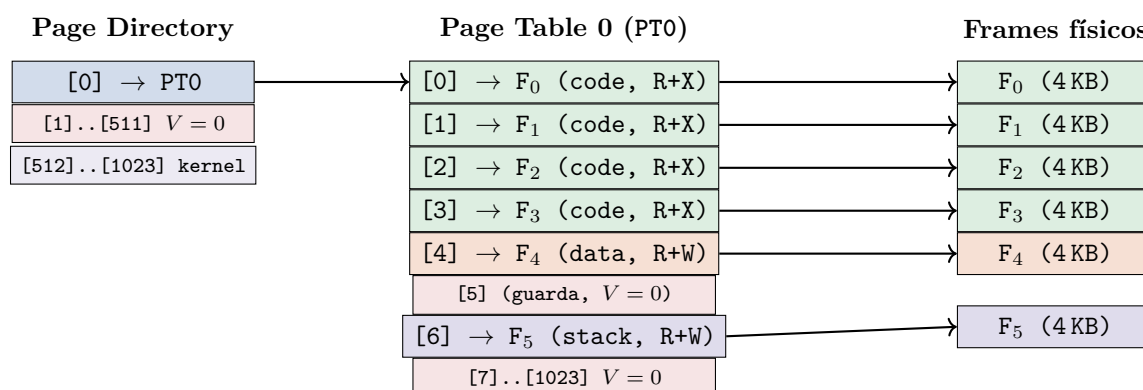


Tabla detallada de entradas válidas

vpn	rango virtual	PT0[idx]	V	permisos	Área
0	[0x0000, 0x1000)	F_0	1	R+X	código (pág. 1)
1	[0x1000, 0x2000)	F_1	1	R+X	código (pág. 2)
2	[0x2000, 0x3000)	F_2	1	R+X	código (pág. 3)
3	[0x3000, 0x4000)	F_3	1	R+X	código (pág. 4, 1 KB padding)
4	[0x4000, 0x5000)	F_4	1	R+W	datos (3 KB + 1 KB pad)
5	[0x5000, 0x6000)	—	0	—	guarda (stack overflow)
6	[0x6000, 0x7000)	F_5	1	R+W	stack

El resto del Page Table 0 (entradas 7 a 1023), así como el resto del Page Directory (entradas 1 a 511 para usuario), quedan con $V = 0$. Las entradas 512 a 1023 del PD están mapeadas al kernel (`KERNBASE = 2 GB` en `xv6 x86`).

Respuesta

El proceso usa **6 frames físicos** (4 de código, 1 de datos, 1 de stack) más **1 página de guarda** (sin frame asociado). Todo el layout cabe en los primeros 28 KB del espacio virtual: una sola entrada del Page Directory (`PD[0]`) que apunta a un único Page Table (`PT0`) con 7 entradas relevantes (de las cuales una es inválida).

8. Ejercicio 8 — Segmentación y paginado

Enunciado. Marcar las respuestas verdaderas.

8.a — “Los segmentos representan porciones lógicas de un proceso”

Respuesta

V. Es justamente la idea fundamental de la segmentación: dividir el espacio de direcciones de un proceso en porciones *con significado lógico* (código, datos, stack, heap, librería compartida...), cada una con su propio tamaño, base y permisos.

8.b — “Un segmento o una página sólo puede contener instrucciones o datos, nunca ambos”

Respuesta

F. Una **página** no distingue “instrucciones de datos”: es una unidad puramente física (4 KB de memoria). Una sola página puede contener instrucciones y datos mezclados (de hecho, el último byte de un binario regularmente combina código y datos en la misma página por alineamiento).

En **segmentación pura**, sí hay segmentos especializados (segmento de código vs. segmento de datos), pero incluso ahí el segmento de código puede contener constantes (`.rodata`). La afirmación absoluta es falsa.

8.c — “El paginado es comúnmente usado para implementar memoria virtual”

Respuesta

V. La paginación es *la* técnica estándar para memoria virtual en SO modernos. Sus ventajas: páginas de tamaño fijo (swap simple), sin fragmentación externa, soporte natural para demand paging y copy-on-write.

8.d — “En x86 las páginas pueden ser de un nivel o de dos niveles”

Respuesta

V. x86 32-bit admite dos modos de paginación:

- **2 niveles:** PD + PT, páginas de **4 KB**. La configuración estándar.
- **1 nivel:** sólo el PD con páginas de **4 MB** (extensión PSE, *Page Size Extension*). Cada entrada del PD apunta directamente a una página de 4 MB. Útil para mapear regiones grandes del kernel.

x86 64-bit usa 4 ó 5 niveles, pero en el contexto de x86 32-bit (que es lo que cubre la cátedra), la afirmación es V.

8.e — “En xv6 (original) al ocurrir un page fault, el proceso finaliza (exit)”

Respuesta

V. xv6 *no implementa* demand paging, COW ni crecimiento dinámico del stack. Cualquier page fault en modo usuario es tratado como un acceso ilegal: `trap.c` marca `p->killed = 1`, lo que provoca que el proceso termine en su próximo `trapret`.

En SO más sofisticados (Linux), un page fault es un *mecanismo útil* para implementar varias optimizaciones; sólo se mata el proceso si la dirección no pertenece a ningún VMA. Pero en xv6 original, **todo** page fault de usuario es fatal.

Resumen Ej. 8

Ítem	Respuesta	Marcar como verdadera
a	V	✓
b	F	
c	V	✓
d	V	✓
e	V	✓

9. Ejercicio 9 — Una sola tabla de páginas en x86 para 4 GB

Enunciado. Si se usa una única tabla de páginas en x86 (32 bits). Para describir 4 GB se requiere que:

- (a) sus entradas describan páginas de **4 KB**.
- (b) sus entradas describan páginas de **4 MB**.

Análisis

“Una única tabla de páginas” significa paginación de *un solo nivel* (sin Page Directory + Page Table; sólo una tabla plana).

Para cubrir los 2^{32} bytes (4 GB) del espacio de direcciones, las entradas de la tabla y el offset deben sumar 32 bits. Si cada entrada cubre una página de tamaño P , hace falta una tabla con $2^{32}/P$ entradas.

Caso (a) páginas de 4 KB: la tabla tendría $2^{32}/2^{12} = 2^{20} = 1\,048\,576$ entradas. Cada entrada ocupa 4 bytes $\rightarrow$ **4 MB** de tabla por proceso. Además, en x86 esto no se hace “en una sola tabla”; el hardware del procesador requiere PD + PT (2 niveles) para páginas de 4 KB.

Caso (b) páginas de 4 MB: la tabla tendría $2^{32}/2^{22} = 2^{10} = 1024$ entradas. Cada entrada ocupa 4 bytes $\rightarrow$ **4 KB** de tabla por proceso (cabe en una sola página). **x86 lo soporta de forma nativa** mediante el bit PS (Page Size) en las entradas del Page Directory: si está encendido, la entrada apunta directamente a una página de 4 MB en lugar de a otro nivel.

Respuesta

La respuesta correcta es **(b) páginas de 4 MB**. Una única tabla de 1024 entradas (4 KB de tamaño) cubre los 4 GB con páginas de 4 MB cada una. Páginas de 4 KB exigirían 1 M de entradas (4 MB de tabla), y x86 no soporta esa configuración con una sola tabla; requiere paginación de dos niveles.

10. Ejercicio 10 — Concepto de memoria virtual

Enunciado. Describir el concepto de memoria virtual.

Definición

La **memoria virtual** es una abstracción provista conjuntamente por el hardware (MMU) y el SO que desacopla la *dirección lógica* (la que un proceso emite cuando lee o escribe memoria) de la *dirección física* (la que en realidad selecciona una celda de la RAM).

Esto permite que cada proceso tenga su **propio espacio de direcciones virtuales** independiente, que puede ser *más grande* que la RAM física disponible y que el SO administre dinámicamente.

Mecanismo básico

1. Cada proceso tiene su propia **tabla de páginas** que el SO mantiene.
2. Cuando el proceso emite una dirección virtual, la **MMU** la consulta en la tabla y la traduce a la dirección física correspondiente.
3. Si la página no está en RAM ($V = 0$), la MMU genera un *page fault*. El SO atiende el fault, decide qué hacer (cargar la página de disco, asignar una nueva, copiar, matar al proceso, etc.) y reanuda la instrucción.

Funcionalidades que habilita

1) Procesos más grandes que la RAM. Swapping: páginas no usadas recientemente se mueven a una zona de *swap* en disco. Cuando se vuelven a referenciar, el SO las trae de vuelta (vía page fault). Permite ejecutar procesos que en conjunto exceden la RAM, al costo de latencia de disco.

2) Demand paging. Las páginas se cargan en RAM **sólo cuando se accede a ellas por primera vez**. Esto acelera el arranque de programas grandes: no hay que leer el ejecutable entero al disco al hacer **exec**, sólo las páginas que se ejecutan.

3) Aislamiento entre procesos. Como cada proceso ve solo su tabla de páginas, no puede acceder a la memoria de otro. Es la base de la seguridad multi-usuario.

4) Copy-on-Write (COW). **fork** no copia la memoria del padre. Las páginas se comparten físicamente con permiso de **lectura**; el primer **write** dispara un page fault que el kernel resuelve copiando sólo la página afectada. Hace **fork+exec** extremadamente eficiente.

5) Memory-mapped files (mmap). Un archivo se “mapea” al espacio de direcciones del proceso. Las páginas se cargan bajo demanda; lecturas y escrituras se traducen automáticamente en operaciones sobre el archivo. Es la base de cargar ejecutables eficientemente.

6) Memoria compartida entre procesos. Dos procesos pueden mapear las mismas páginas físicas a (posiblemente distintas) direcciones virtuales. Es la base de **shm**, IPC de alto rendimiento y librerías compartidas.

7) Páginas de guarda. El SO puede dejar páginas con $V = 0$ entre regiones críticas (e.g., entre el stack y el heap) para detectar overflows inmediatamente.

8) Address Space Layout Randomization (ASLR). La MMU permite cargar el binario en direcciones aleatorias en cada ejecución, dificultando ataques que dependen de direcciones conocidas.

Costos

- **Overhead de traducción:** cada acceso pasa por la MMU. Para mitigar, las CPUs tienen un **TLB** (cache de traducciones).
- **Page faults** cuestan miles de ciclos. Si se dispara mucho (*thrashing*, $\text{working set} > \text{RAM}$), el sistema se cae a pique.
- **Memoria de la tabla de páginas** (algunos MB por proceso en x86-64), aunque el árbol jerárquico mitiga cuando hay grandes huecos.

Respuesta

Memoria virtual es la abstracción que da a cada proceso un espacio de direcciones lógicas independiente, posiblemente mayor que la RAM física, gestionado por el SO con ayuda de la MMU. Sus funcionalidades clave: aislamiento de procesos, swapping, *demand paging*, *copy-on-write*, *memory-mapped files*, memoria compartida entre procesos, ASLR y páginas de guarda.

La idea unificadora: la tabla de páginas es una indirección flexible entre lo que el proceso “ve” y lo que la RAM “es”, y el SO la usa para implementar todas las features anteriores con un único mecanismo (resolver el page fault apropiadamente).

Resumen del parcial

#	Tema	Respuesta sintética
1	Diseño de SO	a:V, b:F, c:V, d:V (microkernel distribuible), e:F
2	Gestión de procesos	a:V, b:F (zombie ya finalizó), c:F (exec no crea), d:F , e:F
3	fork + exec	Ventana para configurar el ambiente del hijo: redirecciones, pipes, env, privilegios, dir. de trabajo, signal handlers
4	xv6	a:V (spinlock), b:F (sin monitor), c:V (sleep/wakeup), d:F, e:F
5	Solución a deadlock	Orden global de adquisición; alternativas: lock único o try-lock+backoff
6	Traducción de direcciones	a:0x02000320, b:0x0A000FFA, c:0x05000200; frame 3: [0x00002000, 0x00002FFF]
7	Mapa xv6 (15K + 3K)	PD[0] → PT0; PT0[0..3]=code, [4]=data, [5]=guarda (V=0), [6]=stack
8	Segmentación/paginado	a:V, b:F, c:V, d:V, e:V
9	Tabla única para 4 GB	b: páginas de 4 MB (1024 entradas, 4 KB de tabla)
10	Memoria virtual	Abstracción para aislamiento, swap, demand paging, COW, mmap, shared mem, ASLR, guard pages

Importante

Correcciones respecto de las respuestas originales:

1. **Ej 1.d:** era **V** (microkernel distribuible), no “no sé”.
2. **Ej 2.d, 2.e:** ambas **F**; estaban incompletas.
3. **Ej 4.b:** **F** (xv6 no tiene monitores como construcción explícita; tiene los componentes por separado).
4. **Ej 6.d (rango del frame 3):** corregido a **[0x00002000, 0x00002FFF]**. El rango original **[0x00400000, 0x0040BFFF]** era inconsistente con las respuestas (a), (b) y (c).